



Semáforos



Un semáforo es una **primitiva de control de la concurrencia** basada en bloqueos. Permite resolver dos tipos de problemas: **exclusión mutua** y **sincronización**.

Explicación

Dentro de la asignatura tenemos que distinguir entre el semáforo teórico y el semáforo en términos de implementación.

Semáforo teórico

Tal como define Palma, un semáforo es:

- Una **variable entera** que toma valores **NO NEGATIVOS** `S`.
- Una **cola de procesos bloqueados** asociada a dicha variable.
 - Por tanto, el semáforo es una solución más eficiente que las de espera ocupada, ya que se basa en bloqueos, no en espera ocupada.
- Dos operaciones atómicas: `wait()` y `signal()`.

`Wait(S)`

```
wait(S):  
  if S > 0:  
    S = S - 1  
  else:  
    bloquear proceso en cola asociada a S
```

Esta es una OPERACIÓN BLOQUEANTE sobre el proceso que lo ejecuta.

`Signal(S)`

```
signal(S):  
  if hay procesos bloqueados:  
    despertar uno de ellos  
  else:  
    S = S + 1
```

Sobre la implementación de la cola no necesitamos saber nada.

Ejemplo en pseudocódigo.

```
sem mutex = 1 // semáforo binario  
  
Proceso i:  
  while true do  
    wait(mutex)  
    -- SECCIÓN CRÍTICA --  
    -- (por ejemplo, acceso a un buffer compartido) --  
    signal(mutex)  
    -- SECCIÓN NO CRÍTICA --  
  end while
```

Tan solo recordar que en lenguajes como Java y C++ no podemos garantizar prioridades. Por lo que la operación de Signal despertará a UN PROCESO CUALQUIERA de los que estén esperando.

En un semáforo binario, si `S = 1` y hacemos más `signal(S)`, **las señales se pierden** (Palma lo recalca también: un binario no almacena señales).

Tenemos 2 tipos de semáforos.

Semáforos Binarios

S toma los valores 0 o 1.

Nos ayudan a implementar exclusión mutua.

Podemos simularlo usando un monitor:

Semaforo mutex

inicio:

```
mutex.init(1)
```

```
-- crear procesos ...
```

fin

Proceso i:

```
while true do
```

```
  mutex.waitSem()
```

```
    -- SECCIÓN CRÍTICA --
```

```
  mutex.signalSem()
```

```
    -- SECCIÓN NO CRÍTICA --
```

```
end while
```

Semáforos Generales

S toma valores enteros NO NEGATIVOS.

Nos sirven para conseguir sincronización y exclusión mutua.

Para **solo garantiza exclusión mutua si NUNCA supera 1** (vamos, cuando funcionan como un binario, solo con valores 0, 1, e inicializados a 1).

- Un semáforo general inicializado a 1 **no es automáticamente un binario**.
- Solo actúa como tal si **ninguna ejecución provoca que $S > 1$** .
- Cuando $S \geq 2$, dos procesos pueden pasar por `wait()` sin bloquearse → **exclusión mutua rota**. Sin embargo, la exclusión mutua **se rompe en el momento en que el semáforo general alcanza el valor 2 o superior**. Porque entonces **dos hilos podrían hacer `wait()` sin bloquearse**, entrando simultáneamente en la sección crítica.

Ejemplo de un semáforo general garantizando exclusión mutua

```
sem g = 1; // inicializamos a uno
```

```
wait(g); // g = 0 → entro
```

```
...  
signal(g); // g = 1 → permitido
```

Mientras solo haya **un signal por cada wait**, todo bien.

Ejemplo de un semáforo general que rompe la exclusión mutua

```
sem g = 1; // inicializamos a uno  
  
signal(g); // g = 2 → peligro mortal  
wait(g); // g = 1 → hilo A entra  
wait(g); // g = 0 → hilo B entra
```

Podemos simularlo usando un monitor:

```
monitor Semaforo  
  
  int S  
  condition cola  
  
  -- Constructor  
  procedure init(valorInicial)  
    S = valorInicial  
  end procedure  
  
  procedure waitSem()  
    while S == 0 do  
      wait(col)  -- el proceso se bloquea en la condición  
    end while  
    S = S - 1  
  end procedure  
  
  procedure signalSem()  
    S = S + 1  
    signal(col)  -- despierta a un proceso (si hay)  
  end procedure
```

end monitor

Implementación

Java API Standart

A este nivel de la API, **NO TENEMOS SEMÁFOROS**.

Tan solo podemos SIMULAR su comportamiento.

La idea es replicar el semáforo teórico con:

- Una variable `int S`
- Un monitor (`synchronized`)
- Las operaciones `wait()` y `notify()` / `notifyAll()` del propio objeto (definidas en `Object`).

```
public class Semaforo {
    private int S;
    private final Object lock = new Object();

    public Semaforo(int valorInicial) {
        this.S = valorInicial;
    }

    public void waitSem() throws InterruptedException {
        synchronized(lock) {
            while (S == 0) {
                lock.wait(); // hilo se bloquea y libera el cerrojo
            }
            S = S - 1;
        }
    }

    public void signalSem() {
        synchronized(lock) {
```

```

        S = S + 1;
        lock.notify(); // despierta a un hilo cualquiera
    }
}
}

```

De esta forma, con una **entera + monitor + wait/notify**, simulamos el comportamiento de un semáforo teórico en la API estándar, sin necesidad de la clase `Semaphore` de la API de alto nivel.

Java API Alto Nivel

En este nivel de la API **SÍ TENEMOS** una implementación directa de semáforos en `java.util.concurrent.Semaphore`.

```

Semaphore sem = new Semaphore(N); // Semáforo general
Semaphore sem = new Semaphore(1); // Semáforo binario

```

Las operaciones cambian de nombre:

- `acquire()` ≈ `wait()`
- `release()` ≈ `signal()`

(Este cambio es necesario porque `wait()` y `signal()` son operaciones que tiene todo objeto que herede de `Object`, y como crearemos objetos de la clase `Semaphore`, esos métodos ya existirían para el objeto)

`Acquire()` es bloqueante y **puede lanzar** `InterruptedException`, por lo que siempre tenemos que usarlo dentro de un `try/catch`.

```

Semaphore mutex = new Semaphore(1);

mutex.acquire(); // bloqueante
// sección crítica
mutex.release();

```

Aunque también tenemos un método del `acquire` que es NO BLOQUEANTE

```

if (mutex.tryAcquire()) {
    // sección crítica
    mutex.release();
} else {
    // no había permiso
}

```

Lo único importante a saber es que los métodos **ESTÁN PARAMETRIZADOS**.

Podemos pasarle un número y hacer, por ejemplo, `acquire(5)`, restando de ese modo 5 a la variable S.

```

acquire(5) // intenta hacer S = S - 5 (bloquea si no hay suficientes permisos).

release(5) // hace S = S + 5 y puede despertar hasta 5 hilos bloqueados.

```

C++

Desde **C++20** disponemos de `<semaphore>`:

- `std::binary_semaphore` → semáforo binario.
- `std::counting_semaphore<N>` → semáforo general con capacidad máxima `N`.

Operaciones:

- `acquire()` ≈ `wait()`
- `release()` ≈ `signal()`

También existen variantes `try_acquire()`, `try_acquire_for()`, etc.

```

// Usamos semáforos de C++20

#include <semaphore>
#include <thread>
#include <iostream>

std::binary_semaphore mutex(1); // valor inicial 1

```

```

int recurso = 0;

void tarea(int id) {
    mutex.acquire(); // wait
    // sección crítica
    int local = recurso;
    local++;
    recurso = local;
    std::cout << "Hilo " << id << " incrementa a " << recurso << "\n";
    // fin sección crítica
    mutex.release(); // signal
}

int main() {
    std::thread t1(tarea, 1);
    std::thread t2(tarea, 2);

    t1.join();
    t2.join();
}

```

En sistemas sin C++20, podemos **simularlos** con:

- `std::mutex` + `std::unique_lock<std::mutex>`
- `std::condition_variable`
- contador entero interno.

```

#include <mutex>
#include <condition_variable>

class Semaphore {
private:
    std::mutex m;
    std::condition_variable cv;

```



```

int count; // valor del semáforo (>= 0)

public:
    explicit Semaphore(int initial = 0)
        : count(initial) {}

    void acquire() {
        std::unique_lock<std::mutex> lk(m);
        while (count == 0) { // condición de guarda
            cv.wait(lk); // se bloquea y libera el mutex
        }
        --count; // sección crítica del semáforo
    }

    void release() {
        std::unique_lock<std::mutex> lk(m);
        ++count;
        cv.notify_one(); // despierta a un hilo cualquiera
    }
};

```